

Keywords: LLM reasoning • formal verification • SMT solver • chain-of-thought

LogicTrack

Auditing LLM Reasoning Trajectories with Formal Logic Solvers and Step-Level Backtracking Rewards

By Jingyu Hu, Shu Yang, Weiru Liu & Di Wang

arXiv:2609.21492 • September 2026

LogicTrack auto-formalizes each chain-of-thought step into an SMT-LIB specification, verifies it with a Z3-class solver, and converts the diagnostics into a Solver-Based Backtracking Reward that guides both inference-time tree search and supervised fine-tuning—improving step-level verifiability across 8 benchmarks and 7 LLMs.

AT A GLANCE

Problem: RLVR optimises final-answer correctness but leaves the logical validity of intermediate reasoning steps unverified and potentially unsound.

Why it matters: Models that reach correct answers via logically invalid steps are brittle, untrustworthy, and opaque to auditors.

Method: Decompose each CoT step into context, assumptions, and conclusion; auto-formalize to SMT-LIB; verify with a theorem prover; combine into a Solver-Based Backtracking Reward (SBR).

Result: Step-level verifiability improves 15–30 pp; final accuracy preserved or improved across 8 benchmarks, 7 LLMs.

Evidence: Systematic evaluation on GSM8K, MATH, ARC, LogiQA, FOLIO, ProntoQA, and custom multi-step deduction suites.

The Big Picture

Large language model reasoning has been transformed by chain-of-thought (CoT) prompting and reinforcement learning with verifiable rewards (RLVR). Models now generate extended thinking traces before answering, and these traces substantially improve performance on complex tasks. But the training signal in RLVR is a binary or scalar reward over the final answer—not over the intermediate steps.

This creates a subtle failure mode: models can learn to produce long, plausible-looking reasoning chains that are internally inconsistent or logically invalid, while still arriving at correct answers through pattern-matching or lucky leaps. Such traces are unauditable, brittle to dis-

tributional shift, and dangerous in high-stakes domains where the reasoning process—not just the answer—must be verified.

LogicTrack addresses this gap with a neuro-symbolic framework that provides free, scalable, per-step logical verification as a training and inference-time signal.

Why Existing Approaches Fall Short

Outcome-based RLVR (GRPO, PPO-R1) provides a single reward per trajectory, giving no credit for valid intermediate steps or penalty for invalid ones. A model is rewarded equally for a logically sound path and a lucky leap.

Process reward models (PRMs) score intermediate steps but are trained discriminatively on expensive human labels and do not carry logical soundness guarantees—they reflect human preferences, not logical validity.

Self-consistency and majority voting increase answer reliability through repeated sampling but do not verify any individual trace.

Direct SMT encoding cannot process natural language without a formalization layer, which has historically required domain-specific engineering and failed to generalize.

LogicTrack pairs a neural auto-formalization module (an LLM translator) with a classical SMT solver, getting the generalization of neural models and the soundness guarantees of symbolic reasoning.

Core Mechanism

Step Decomposition. Each natural-language step s_k is decomposed into three fields:

1. **Context:** facts established by prior steps $s_1, \dots, s_{k-1}$
2. **Background assumptions:** domain knowledge or definitions
3. **Conclusion:** the claimed logical consequence of 1 and 2

Auto-formalization. An LLM translator maps each triple into an SMT-LIB specification covering linear arithmetic, equality, and propositional logic—the fragment that covers the large majority of mathematical and logical reasoning steps.

Solver Verification. A Z3-class SMT solver checks the specification and returns one of: *syntax error*, *satisfiable*, *entailed* (conclusion follows from context + assumptions), *refuted* (conclusion contradicts them), or *undecided* (solver timeout).

Solver-Based Backtracking Reward (SBR). SBR combines:

- A *premise-fidelity* score measuring how faithfully the

SMT context matches prior step conclusions

- A *solver verification score*: *entailed* > *satisfiable* > *undecided* > *refuted*

Inference-time Backtracking. SBR guides tree search: when the current path stalls, the search backtracks to the step with the lowest SBR score and regenerates from that branch.

SFT data construction. Successful backtracking trajectories are formatted as SFT examples, teaching fine-tuned models to internalise step-wise auditing without requiring the solver at inference time.

Technical Formulation

Let trajectory $\tau = (s_1, \dots, s_T, a)$ where a is the final answer. Each step s_k maps to an SMT triple $(\text{ctx}_k, \text{bg}_k, \text{conc}_k)$.

The SMT validity check:

$$\text{SMT-check} : \text{ctx}_k \wedge \text{bg}_k \models \text{conc}_k$$

Step-level SBR:

$$\text{SBR}(s_k) = \alpha \cdot \text{Fid}(s_k) + (1 - \alpha) \cdot \text{SolverScore}(s_k)$$

Trajectory-level SBR (used as training reward):

$$\text{SBR}(\tau) = \frac{1}{T} \sum_{k=1}^T \text{SBR}(s_k) \cdot \mathbf{1}[a = a^*]$$

Backtrack point selection:

$$k^* = \arg \min_k \text{SBR}(s_k)$$

The search then regenerates $s_{k^*}, \dots, s_T$ from the new branch.

Learning or Inference Procedure

Inference-time:

1. Generate initial trajectory τ .
2. Score each step with SBR; if $\text{SBR}(\tau) < \delta$ backtrack to k^* and regenerate.
3. Repeat until answer correct or budget exhausted.

Training-time SFT:

1. Collect successful backtracking trajectories.
2. Format with explicit `<backtrack>` markers.
3. Fine-tune base model; solver not required at inference.

What the Guarantee Says

For steps in the linear-arithmetic and propositional-logic fragment, the SMT check is sound and (conditionally)

complete: an *entailed* verdict guarantees the conclusion logically follows from the stated context and assumptions. Steps marked *refuted* are provably invalid under the formalized premises. The *undecided* category handles solver incompleteness gracefully.

Experimental Findings

- **8 benchmarks:** GSM8K, MATH, ARC-Challenge, LogiQA, FOLIO, ProntoQA, HellaSwag-hard, custom multi-step deduction
- **7 LLMs:** 7B to 70B parameter range
- Step-level verifiability improves by 15–30 percentage points across all model families
- Final answer accuracy improves on 5 of 8 benchmarks; negligible change on the remaining 3
- SFT on backtracking traces provides lasting improvements without runtime solver dependency
- Largest gains on tasks requiring deductive chaining (FOLIO, ProntoQA)

Ablations and Interpretation

SBR vs. outcome-only reward: Outcome reward improves final accuracy but not step-level verifiability; SBR improves both, confirming that process supervision is strictly more informative.

Backtracking vs. repeated sampling: At equal inference budget, backtracking achieves higher accuracy by directing search toward logical faults rather than sampling independently.

Auto-formalization quality: $\approx 85\%$ of steps are successfully formalized; failures are handled as *undecided* and penalised only lightly, preventing over-penalisation of complex steps that exceed the SMT-LIB fragment.

α **sensitivity:** Performance is stable for $\alpha \in [0.3, 0.7]$, with a slight preference for $\alpha = 0.5$.

Reference

Jingyu Hu, Shu Yang, Weiru Liu, Di Wang. **LogicTrack: Auditing Reasoning Trajectories of Large Language Models with Formal Logic Solvers.** arXiv:2609.21492, September 2026.
<https://arxiv.org/abs/2609.21492>